React & Next.js Guide with Examples

Table of Contents

1. Core React Concepts
2. React Hooks
3. State Management
4. Component Patterns
5. React Router
6. Next.js Rendering Strategies
7. Next.js App Router
8. Next.js Features
9. Data Fetching
10. CSS & Styling
11. Performance Optimization
12. Form Handling
13. TypeScript with React
14. Testing
15. Build Tools & Deployment

Core React Concepts

1. Components & Props

What are Components?

Components are independent, reusable pieces of UI. They accept inputs (props) and return React elements describing what should appear on the screen.

Functional Components (Modern Approach)

```
// Basic functional component
function Welcome(props) {
  return <h1>Hello, {props.name}</h1>;
}

// Arrow function syntax
const Welcome = (props) => {
  return <h1>Hello, {props.name}</h1>;
};

// Using the component
<Welcome name="Sara" />
```

Class Components (Legacy, but still used)

```
import React, { Component } from 'react';

class Welcome extends Component {
  render() {
    return <h1>Hello, {this.props.name}</h1>;
  }
}
```

Props (Properties)

Props are arguments passed to components, similar to function parameters.

```
// Parent Component
function App() {
  return (
    <div>
      <UserCard
        name="John Doe"
        age={30}
        email="john@example.com"
        isActive={true}
```

```
      />
    </div>
  );
}

// Child Component receiving props
function UserCard(props) {
  return (
    <div className="card">
      <h2>{props.name}</h2>
      <p>Age: {props.age}</p>
      <p>Email: {props.email}</p>
      <span>{props.isActive ? 'Active' : 'Inactive'}</span>
    </div>
  );
}

// Destructuring props (Recommended)
function UserCard({ name, age, email, isActive }) {
  return (
    <div className="card">
      <h2>{name}</h2>
      <p>Age: {age}</p>
      <p>Email: {email}</p>
      <span>{isActive ? 'Active' : 'Inactive'}</span>
    </div>
  );
}
```

Default Props

```
function Button({ text, color = 'blue', size = 'medium' })
{
  return (
    <button className={`btn-${color} btn-${size}`}>
      {text}
    </button>
  );
```

```
}

// Usage
<Button text="Click me" />  // Uses default color and size
<Button text="Submit" color="green" size="large" />
```

Props Drilling Problem

```
// Props drilling - passing props through multiple levels
function App() {
  const user = { name: 'John', role: 'Admin' };
  return <Dashboard user={user} />;
}

function Dashboard({ user }) {
  return <Sidebar user={user} />;
}

function Sidebar({ user }) {
  return <UserProfile user={user} />;
}

function UserProfile({ user }) {
  return <div>{user.name} - {user.role}</div>;
}

// Solution: Use Context API (explained later)
```

Children Props

```
// Container component that wraps other content
function Card({ children, title }) {
  return (
    <div className="card">
      <h2>{title}</h2>
      <div className="card-content">
        {children}
```

```
      </div>
    </div>
  );
}

// Usage
function App() {
  return (
    <Card title="User Information">
      <p>Name: John Doe</p>
      <p>Email: john@example.com</p>
      <button>Edit Profile</button>
    </Card>
  );
}
```

Component Composition

```
// Building complex UIs from smaller components
function Avatar({ user }) {
  return (
    <img
      src={user.avatarUrl}
      alt={user.name}
      className="avatar"
    />
  );
}

function UserInfo({ user }) {
  return (
    <div className="user-info">
      <Avatar user={user} />
      <div>
        <h3>{user.name}</h3>
        <p>{user.email}</p>
      </div>
    </div>
```

```
  );
}

function UserCard({ user, onEdit }) {
  return (
    <div className="user-card">
      <UserInfo user={user} />
      <button onClick={() => onEdit(user.id)}>Edit</button>
    </div>
  );
}
```

React Hooks

2. useState Hook

Basic Usage

```
import { useState } from 'react';

function Counter() {
  // Declare state variable
  const [count, setCount] = useState(0);

  return (
    <div>
      <p>Count: {count}</p>
      <button onClick={() => setCount(count + 1)}>Increment
</button>
      <button onClick={() => setCount(count - 1)}>Decrement
</button>
      <button onClick={() => setCount(0)}>Reset</button>
    </div>
  );
}
```

Multiple State Variables

```
function UserForm() {
  const [name, setName] = useState('');
  const [email, setEmail] = useState('');
  const [age, setAge] = useState(0);

  const handleSubmit = (e) => {
    e.preventDefault();
    console.log({ name, email, age });
  };

  return (
    <form onSubmit={handleSubmit}>
      <input
        value={name}
        onChange={(e) => setName(e.target.value)}
        placeholder="Name"
      />
      <input
        value={email}
        onChange={(e) => setEmail(e.target.value)}
        placeholder="Email"
      />
      <input
        type="number"
        value={age}
        onChange={(e) => setAge(Number(e.target.value))}
        placeholder="Age"
      />
      <button type="submit">Submit</button>
    </form>
  );
}
```

State with Objects

```
function UserProfile() {
  const [user, setUser] = useState({
    name: '',
    email: '',
    address: {
      city: '',
      country: ''
    }
  });

  const updateName = (name) => {
    setUser(prevUser => ({
      ...prevUser,  // Spread previous state
      name: name
    }));
  };

  const updateCity = (city) => {
    setUser(prevUser => ({
      ...prevUser,
      address: {
        ...prevUser.address,
        city: city
      }
    }));
  };

  return (
    <div>
      <input
        value={user.name}
        onChange={(e) => updateName(e.target.value)}
        placeholder="Name"
      />
      <input
        value={user.address.city}
        onChange={(e) => updateCity(e.target.value)}
        placeholder="City"
```

```
      />
    </div>
  );
}
```

State with Arrays

```
function TodoList() {
  const [todos, setTodos] = useState([]);
  const [input, setInput] = useState('');

  const addTodo = () => {
    setTodos([...todos, { id: Date.now(), text: input }]);
    setInput('');
  };

  const deleteTodo = (id) => {
    setTodos(todos.filter(todo => todo.id !== id));
  };

  const updateTodo = (id, newText) => {
    setTodos(todos.map(todo =>
      todo.id === id ? { ...todo, text: newText } : todo
    ));
  };

  return (
    <div>
      <input
        value={input}
        onChange={(e) => setInput(e.target.value)}
      />
      <button onClick={addTodo}>Add Todo</button>

      <ul>
        {todos.map(todo => (
          <li key={todo.id}>
            {todo.text}
```

```
            <button onClick={() => deleteTodo(todo.id)}>Del
ete</button>
          </li>
        ))}
      </ul>
    </div>
  );
}
```

Lazy Initial State

```
// Expensive computation
function expensiveComputation() {
  console.log('Computing...');
  return Array.from({ length: 1000 }, (_, i) => i);
}

function MyComponent() {
  // ❌ Bad: Function runs on every render
  const [data, setData] = useState(expensiveComputation());

  // ✅ Good: Function runs only once
  const [data, setData] = useState(() => expensiveComputati
on());

  return <div>Data length: {data.length}</div>;
}
```

3. useEffect Hook

Basic Usage

```
import { useEffect } from 'react';

function Example() {
  const [count, setCount] = useState(0);
```

```
  // Runs after every render
  useEffect(() => {
    document.title = `Count: ${count}`;
  });

  return (
    <div>
      <p>Count: {count}</p>
      <button onClick={() => setCount(count + 1)}>Increment
</button>
    </div>
  );
}
```

Dependency Array

```
function UserProfile({ userId }) {
  const [user, setUser] = useState(null);

  // Runs only when userId changes
  useEffect(() => {
    fetch(`/api/users/${userId}`)
      .then(res => res.json())
      .then(data => setUser(data));
  }, [userId]);  // Dependency array

  if (!user) return <div>Loading...</div>;

  return <div>{user.name}</div>;
}
```

Empty Dependency Array (Run Once)

```
function App() {
  const [data, setData] = useState([]);

  // Runs only once after initial render
```

```
  useEffect(() => {
    fetch('/api/data')
      .then(res => res.json())
      .then(data => setData(data));
  }, []);  // Empty array = runs once

  return <div>{/* render data */}</div>;
}
```

Cleanup Function

```
function Timer() {
  const [seconds, setSeconds] = useState(0);

  useEffect(() => {
    const interval = setInterval(() => {
      setSeconds(s => s + 1);
    }, 1000);

    // Cleanup function
    return () => {
      clearInterval(interval);
      console.log('Timer cleaned up');
    };
  }, []);

  return <div>Seconds: {seconds}</div>;
}
```

Multiple Effects

```
function UserDashboard({ userId }) {
  const [user, setUser] = useState(null);
  const [posts, setPosts] = useState([]);

  // Effect 1: Fetch user
  useEffect(() => {
```

```
    fetch(`/api/users/${userId}`)
      .then(res => res.json())
      .then(data => setUser(data));
  }, [userId]);

  // Effect 2: Fetch posts
  useEffect(() => {
    fetch(`/api/users/${userId}/posts`)
      .then(res => res.json())
      .then(data => setPosts(data));
  }, [userId]);

  // Effect 3: Update document title
  useEffect(() => {
    if (user) {
      document.title = `${user.name}'s Dashboard`;
    }
  }, [user]);

  return <div>{/* render UI */}</div>;
}
```

Common useEffect Patterns

```
// Window event listeners
function WindowSize() {
  const [size, setSize] = useState({ width: 0, height: 0
});

  useEffect(() => {
    const handleResize = () => {
      setSize({
        width: window.innerWidth,
        height: window.innerHeight
      });
    };

    window.addEventListener('resize', handleResize);
```

```
    handleResize(); // Initial call

    return () => window.removeEventListener('resize', handl
eResize);
  }, []);

  return <div>Width: {size.width}, Height: {size.height}</d
iv>;
}

// WebSocket connection
function ChatRoom({ roomId }) {
  const [messages, setMessages] = useState([]);

  useEffect(() => {
    const socket = new WebSocket(`ws://api.example.com/room
s/${roomId}`);

    socket.onmessage = (event) => {
      setMessages(prev => [...prev, event.data]);
    };

    return () => {
      socket.close();
    };
  }, [roomId]);

  return <div>{/* render messages */}</div>;
}
```

4. useContext Hook

Creating and Using Context

```
import { createContext, useContext, useState } from 'reac
t';

// 1. Create Context
```

```
const ThemeContext = createContext();

// 2. Create Provider Component
function ThemeProvider({ children }) {
  const [theme, setTheme] = useState('light');

  const toggleTheme = () => {
    setTheme(prevTheme => prevTheme === 'light' ? 'dark' :
'light');
  };

  return (
    <ThemeContext.Provider value={{ theme, toggleTheme }}>
      {children}
    </ThemeContext.Provider>
  );
}

// 3. Create custom hook for easy access
function useTheme() {
  const context = useContext(ThemeContext);
  if (!context) {
    throw new Error('useTheme must be used within ThemeProv
ider');
  }
  return context;
}

// 4. Use in components
function App() {
  return (
    <ThemeProvider>
      <Navbar />
      <MainContent />
    </ThemeProvider>
  );
}
```

```
function Navbar() {
  const { theme, toggleTheme } = useTheme();

  return (
    <nav className={`navbar-${theme}`}>
      <button onClick={toggleTheme}>
        Toggle to {theme === 'light' ? 'dark' : 'light'} mo
de
      </button>
    </nav>
  );
}

function MainContent() {
  const { theme } = useTheme();

  return (
    <main className={`content-${theme}`}>
      <h1>Current theme: {theme}</h1>
    </main>
  );
}
```

Authentication Context Example

```
// AuthContext.js
const AuthContext = createContext();

export function AuthProvider({ children }) {
  const [user, setUser] = useState(null);
  const [loading, setLoading] = useState(true);

  useEffect(() => {
    // Check if user is logged in
    const token = localStorage.getItem('token');
    if (token) {
      fetch('/api/me', {
        headers: { Authorization: `Bearer ${token}` }
```

```
      })
        .then(res => res.json())
        .then(data => setUser(data))
        .finally(() => setLoading(false));
    } else {
      setLoading(false);
    }
  }, []);

  const login = async (email, password) => {
    const response = await fetch('/api/login', {
      method: 'POST',
      body: JSON.stringify({ email, password })
    });
    const data = await response.json();
    localStorage.setItem('token', data.token);
    setUser(data.user);
  };

  const logout = () => {
    localStorage.removeItem('token');
    setUser(null);
  };

  return (
    <AuthContext.Provider value={{ user, login, logout, loa
ding }}>
      {children}
    </AuthContext.Provider>
  );
}

export const useAuth = () => useContext(AuthContext);

// Usage
function ProtectedRoute({ children }) {
  const { user, loading } = useAuth();
```

```
  if (loading) return <div>Loading...</div>;
  if (!user) return <Navigate to="/login" />;

  return children;
}

function LoginForm() {
  const { login } = useAuth();
  const [email, setEmail] = useState('');
  const [password, setPassword] = useState('');

  const handleSubmit = async (e) => {
    e.preventDefault();
    await login(email, password);
  };

  return (
    <form onSubmit={handleSubmit}>
      <input value={email} onChange={e => setEmail(e.targe
t.value)} />
      <input type="password" value={password} onChange={e =
> setPassword(e.target.value)} />
      <button type="submit">Login</button>
    </form>
  );
}
```

5. useRef Hook

DOM References

```
import { useRef, useEffect } from 'react';

function TextInputWithFocus() {
  const inputRef = useRef(null);

  useEffect(() => {
    // Focus input on mount
```

```
    inputRef.current.focus();
  }, []);

  return <input ref={inputRef} type="text" />;
}

// Measuring DOM elements
function MeasureExample() {
  const divRef = useRef(null);

  const measureDiv = () => {
    const rect = divRef.current.getBoundingClientRect();
    console.log('Width:', rect.width);
    console.log('Height:', rect.height);
  };

  return (
    <>
      <div ref={divRef}>Measure me!</div>
      <button onClick={measureDiv}>Get Dimensions</button>
    </>
  );
}
```

Persisting Values Without Re-renders

```
function Timer() {
  const [count, setCount] = useState(0);
  const intervalRef = useRef(null);

  const startTimer = () => {
    if (intervalRef.current) return; // Already running

    intervalRef.current = setInterval(() => {
      setCount(c => c + 1);
    }, 1000);
  };
```

```
  const stopTimer = () => {
    if (intervalRef.current) {
      clearInterval(intervalRef.current);
      intervalRef.current = null;
    }
  };

  useEffect(() => {
    return () => stopTimer(); // Cleanup
  }, []);

  return (
    <div>
      <p>Count: {count}</p>
      <button onClick={startTimer}>Start</button>
      <button onClick={stopTimer}>Stop</button>
    </div>
  );
}
```

Storing Previous Values

```
function usePrevious(value) {
  const ref = useRef();

  useEffect(() => {
    ref.current = value;
  }, [value]);

  return ref.current;
}

// Usage
function Counter() {
  const [count, setCount] = useState(0);
  const prevCount = usePrevious(count);

  return (
```

```
    <div>
      <p>Current: {count}</p>
      <p>Previous: {prevCount}</p>
      <button onClick={() => setCount(count + 1)}>Increment
</button>
    </div>
  );
}
```

6. useMemo Hook

Memoizing Expensive Calculations

```
import { useMemo } from 'react';

function ExpensiveComponent({ numbers }) {
  // Without useMemo: recalculates on every render
  const sum = numbers.reduce((acc, num) => acc + num, 0);

  // With useMemo: only recalculates when 'numbers' changes
  const sum = useMemo(() => {
    console.log('Calculating sum...');
    return numbers.reduce((acc, num) => acc + num, 0);
  }, [numbers]);

  return <div>Sum: {sum}</div>;
}

// Complex filtering example
function ProductList({ products, filters }) {
  const filteredProducts = useMemo(() => {
    console.log('Filtering products...');
    return products.filter(product => {
      if (filters.category && product.category !== filters.
category) {
        return false;
      }
      if (filters.minPrice && product.price < filters.minPr
```

```
ice) {
        return false;
      }
      if (filters.maxPrice && product.price > filters.maxPr
ice) {
        return false;
      }
      return true;
    });
  }, [products, filters]);

  return (
    <div>
      {filteredProducts.map(product => (
        <ProductCard key={product.id} product={product} />
      ))}
    </div>
  );
}
```

Referential Equality

```
function Parent() {
  const [count, setCount] = useState(0);

  // Without useMemo: new object created on every render
  const config = { theme: 'dark', lang: 'en' };

  // With useMemo: same object reference maintained
  const config = useMemo(() => ({
    theme: 'dark',
    lang: 'en'
  }), []);

  return <Child config={config} />;
}

const Child = React.memo(function Child({ config }) {
```

```
  console.log('Child rendered');
  return <div>{config.theme}</div>;
});
```

7. useCallback Hook

Preventing Unnecessary Re-renders

```
import { useCallback } from 'react';

function ParentComponent() {
  const [count, setCount] = useState(0);
  const [items, setItems] = useState([]);

  // Without useCallback: new function created on every ren
der
  const handleClick = () => {
    console.log('Clicked');
  };

  // With useCallback: same function reference maintained
  const handleClick = useCallback(() => {
    console.log('Clicked');
  }, []);

  const addItem = useCallback((item) => {
    setItems(prevItems => [...prevItems, item]);
  }, []);

  return (
    <div>
      <p>Count: {count}</p>
      <button onClick={() => setCount(count + 1)}>Increment
</button>
      <ExpensiveChild onClick={handleClick} />
    </div>
  );
}
```

```
const ExpensiveChild = React.memo(function ExpensiveChild({
onClick }) {
  console.log('Child rendered');
  return <button onClick={onClick}>Click me</button>;
});
```

With Dependencies

```
function SearchComponent() {
  const [query, setQuery] = useState('');
  const [filters, setFilters] = useState({ category: 'all'
});

  // Recreates when query or filters change
  const handleSearch = useCallback(async () => {
    const response = await fetch(
      `/api/search?q=${query}&category=${filters.category}`
    );
    const data = await response.json();
    return data;
  }, [query, filters]);

  return (
    <div>
      <input value={query} onChange={e => setQuery(e.targe
t.value)} />
      <SearchResults onSearch={handleSearch} />
    </div>
  );
}
```

8. useReducer Hook

Basic Usage

```jsx
import { useReducer } from 'react';

// 1. Define initial state
const initialState = { count: 0 };

// 2. Define reducer function
function reducer(state, action) {
  switch (action.type) {
    case 'increment':
      return { count: state.count + 1 };
    case 'decrement':
      return { count: state.count - 1 };
    case 'reset':
      return initialState;
    default:
      throw new Error('Unknown action type');
  }
}

// 3. Use in component
function Counter() {
  const [state, dispatch] = useReducer(reducer, initialStat
e);

  return (
    <div>
      <p>Count: {state.count}</p>
      <button onClick={() => dispatch({ type: 'increment'
})}>+</button>
      <button onClick={() => dispatch({ type: 'decrement'
})}>-</button>
      <button onClick={() => dispatch({ type: 'reset' })}>R
eset</button>
    </div>
  );
}
```

Complex State Management

```
const initialState = {
  user: null,
  loading: false,
  error: null,
  posts: []
};

function reducer(state, action) {
  switch (action.type) {
    case 'FETCH_START':
      return {
        ...state,
        loading: true,
        error: null
      };
    case 'FETCH_SUCCESS':
      return {
        ...state,
        loading: false,
        user: action.payload.user,
        posts: action.payload.posts
      };
    case 'FETCH_ERROR':
      return {
        ...state,
        loading: false,
        error: action.payload
      };
    case 'ADD_POST':
      return {
        ...state,
        posts: [...state.posts, action.payload]
      };
    case 'DELETE_POST':
      return {
        ...state,
```

```
        posts: state.posts.filter(post => post.id !== actio
n.payload)
      };
    default:
      return state;
  }
}

function UserDashboard() {
  const [state, dispatch] = useReducer(reducer, initialStat
e);

  useEffect(() => {
    dispatch({ type: 'FETCH_START' });

    fetch('/api/user/dashboard')
      .then(res => res.json())
      .then(data => {
        dispatch({
          type: 'FETCH_SUCCESS',
          payload: data
        });
      })
      .catch(error => {
        dispatch({
          type: 'FETCH_ERROR',
          payload: error.message
        });
      });
  }, []);

  const addPost = (post) => {
    dispatch({ type: 'ADD_POST', payload: post });
  };

  const deletePost = (postId) => {
    dispatch({ type: 'DELETE_POST', payload: postId });
  };
```

```
  if (state.loading) return <div>Loading...</div>;
  if (state.error) return <div>Error: {state.error}</div>;

  return (
    <div>
      <h1>{state.user?.name}</h1>
      <PostList posts={state.posts} onDelete={deletePost} /
>
    </div>
  );
}
```

Form Management with useReducer

```
const initialFormState = {
  name: '',
  email: '',
  password: '',
  confirmPassword: '',
  errors: {}
};

function formReducer(state, action) {
  switch (action.type) {
    case 'UPDATE_FIELD':
      return {
        ...state,
        [action.field]: action.value,
        errors: {
          ...state.errors,
          [action.field]: undefined
        }
      };
    case 'SET_ERRORS':
      return {
        ...state,
        errors: action.payload
```

```
      };
    case 'RESET_FORM':
      return initialFormState;
    default:
      return state;
  }
}

function RegistrationForm() {
  const [state, dispatch] = useReducer(formReducer, initial
FormState);

  const handleChange = (field, value) => {
    dispatch({ type: 'UPDATE_FIELD', field, value });
  };

  const handleSubmit = (e) => {
    e.preventDefault();

    const errors = {};
    if (!state.name) errors.name = 'Name is required';
    if (!state.email) errors.email = 'Email is required';
    if (state.password !== state.confirmPassword) {
      errors.confirmPassword = 'Passwords do not match';
    }

    if (Object.keys(errors).length > 0) {
      dispatch({ type: 'SET_ERRORS', payload: errors });
      return;
    }

    // Submit form
    console.log('Form submitted:', state);
  };

  return (
    <form onSubmit={handleSubmit}>
      <input
```

```
        value={state.name}
        onChange={(e) => handleChange('name', e.target.valu
e)}
        placeholder="Name"
      />
      {state.errors.name && <span>{state.errors.name}</span
>}

      <input
        value={state.email}
        onChange={(e) => handleChange('email', e.target.val
ue)}
        placeholder="Email"
      />
      {state.errors.email && <span>{state.errors.email}</sp
an>}

      <button type="submit">Register</button>
    </form>
  );
}
```

9. Custom Hooks

Creating Reusable Logic

```
// useLocalStorage hook
function useLocalStorage(key, initialValue) {
  const [storedValue, setStoredValue] = useState(() => {
    try {
      const item = window.localStorage.getItem(key);
      return item ? JSON.parse(item) : initialValue;
    } catch (error) {
      console.error(error);
      return initialValue;
    }
  });
```

```
  const setValue = (value) => {
    try {
      const valueToStore = value instanceof Function ? valu
e(storedValue) : value;
      setStoredValue(valueToStore);
      window.localStorage.setItem(key, JSON.stringify(value
ToStore));
    } catch (error) {
      console.error(error);
    }
  };

  return [storedValue, setValue];
}

// Usage
function App() {
  const [name, setName] = useLocalStorage('name', '');
  const [theme, setTheme] = useLocalStorage('theme', 'ligh
t');

  return (
    <div>
      <input value={name} onChange={e => setName(e.target.v
alue)} />
      <button onClick={() => setTheme(theme === 'light' ?
'dark' : 'light')}>
        Toggle Theme
      </button>
    </div>
  );
}
```

useFetch Hook

```
function useFetch(url) {
  const [data, setData] = useState(null);
  const [loading, setLoading] = useState(true);
```

```
  const [error, setError] = useState(null);

  useEffect(() => {
    setLoading(true);
    setError(null);

    fetch(url)
      .then(response => {
        if (!response.ok) throw new Error('Network response
was not ok');
        return response.json();
      })
      .then(data => {
        setData(data);
        setLoading(false);
      })
      .catch(error => {
        setError(error.message);
        setLoading(false);
      });
  }, [url]);

  return { data, loading, error };
}

// Usage
function UserProfile({ userId }) {
  const { data, loading, error } = useFetch(`/api/users/${u
serId}`);

  if (loading) return <div>Loading...</div>;
  if (error) return <div>Error: {error}</div>;

  return (
    <div>
      <h1>{data.name}</h1>
      <p>{data.email}</p>
    </div>
```

```
  );
}
```

useDebounce Hook

```
function useDebounce(value, delay) {
  const [debouncedValue, setDebouncedValue] = useState(valu
e);

  useEffect(() => {
    const handler = setTimeout(() => {
      setDebouncedValue(value);
    }, delay);

    return () => {
      clearTimeout(handler);
    };
  }, [value, delay]);

  return debouncedValue;
}

// Usage
function SearchComponent() {
  const [searchTerm, setSearchTerm] = useState('');
  const debouncedSearchTerm = useDebounce(searchTerm, 500);

  useEffect(() => {
    if (debouncedSearchTerm) {
      // Perform search
      fetch(`/api/search?q=${debouncedSearchTerm}`)
        .then(res => res.json())
        .then(data => console.log(data));
    }
  }, [debouncedSearchTerm]);

  return (
    <input
```

```
      value={searchTerm}
      onChange={e => setSearchTerm(e.target.value)}
      placeholder="Search..."
    />
  );
}
```

useToggle Hook

```
function useToggle(initialValue = false) {
  const [value, setValue] = useState(initialValue);

  const toggle = useCallback(() => {
    setValue(v => !v);
  }, []);

  return [value, toggle];
}

// Usage
function Modal() {
  const [isOpen, toggleOpen] = useToggle(false);

  return (
    <>
      <button onClick={toggleOpen}>Open Modal</button>
      {isOpen && (
        <div className="modal">
          <h2>Modal Content</h2>
          <button onClick={toggleOpen}>Close</button>
        </div>
      )}
    </>
  );
}
```

useWindowSize Hook

```
function useWindowSize() {
  const [windowSize, setWindowSize] = useState({
    width: undefined,
    height: undefined
  });

  useEffect(() => {
    function handleResize() {
      setWindowSize({
        width: window.innerWidth,
        height: window.innerHeight
      });
    }

    window.addEventListener('resize', handleResize);
    handleResize();

    return () => window.removeEventListener('resize', handl
eResize);
  }, []);

  return windowSize;
}

// Usage
function ResponsiveComponent() {
  const { width, height } = useWindowSize();

  return (
    <div>
      {width < 768 ? (
        <MobileLayout />
      ) : (
        <DesktopLayout />
      )}
    </div>
```

```
  );
}
```

State Management

10. Context API (Advanced)

Complete Example with Multiple Contexts

```
// contexts/AppContext.js
import { createContext, useContext, useReducer } from 'reac
t';

// App Context
const AppContext = createContext();

const initialState = {
  user: null,
  theme: 'light',
  language: 'en',
  notifications: []
};

function appReducer(state, action) {
  switch (action.type) {
    case 'LOGIN':
      return { ...state, user: action.payload };
    case 'LOGOUT':
      return { ...state, user: null };
    case 'SET_THEME':
      return { ...state, theme: action.payload };
    case 'SET_LANGUAGE':
      return { ...state, language: action.payload };
    case 'ADD_NOTIFICATION':
      return {
        ...state,
        notifications: [...state.notifications, action.payl
```

```
oad]
      };
    case 'REMOVE_NOTIFICATION':
      return {
        ...state,
        notifications: state.notifications.filter(n => n.id
!== action.payload)
      };
    default:
      return state;
  }
}

export function AppProvider({ children }) {
  const [state, dispatch] = useReducer(appReducer, initialS
tate);

  const login = (user) => dispatch({ type: 'LOGIN', payloa
d: user });
  const logout = () => dispatch({ type: 'LOGOUT' });
  const setTheme = (theme) => dispatch({ type: 'SET_THEME',
payload: theme });
  const setLanguage = (lang) => dispatch({ type: 'SET_LANGU
AGE', payload: lang });

  const addNotification = (message, type = 'info') => {
    const id = Date.now();
    dispatch({
      type: 'ADD_NOTIFICATION',
      payload: { id, message, type }
    });

    // Auto remove after 5 seconds
    setTimeout(() => {
      dispatch({ type: 'REMOVE_NOTIFICATION', payload: id
});
    }, 5000);
  };
```

```
  const value = {
    ...state,
    login,
    logout,
    setTheme,
    setLanguage,
    addNotification
  };

  return <AppContext.Provider value={value}>{children}</App
Context.Provider>;
}

export const useApp = () => {
  const context = useContext(AppContext);
  if (!context) {
    throw new Error('useApp must be used within AppProvide
r');
  }
  return context;
};
```

11. Redux Toolkit

Store Setup

```
// store/store.js
import { configureStore } from '@reduxjs/toolkit';
import counterReducer from './counterSlice';
import userReducer from './userSlice';

export const store = configureStore({
  reducer: {
    counter: counterReducer,
    user: userReducer
  }
});
```

```jsx
// main.jsx or index.jsx
import { Provider } from 'react-redux';
import { store } from './store/store';

ReactDOM.createRoot(document.getElementById('root')).render
(
  <Provider store={store}>
    <App />
  </Provider>
);
```

Creating Slices

```jsx
// store/counterSlice.js
import { createSlice } from '@reduxjs/toolkit';

const counterSlice = createSlice({
  name: 'counter',
  initialState: {
    value: 0
  },
  reducers: {
    increment: (state) => {
      state.value += 1; // Redux Toolkit uses Immer interna
lly
    },
    decrement: (state) => {
      state.value -= 1;
    },
    incrementByAmount: (state, action) => {
      state.value += action.payload;
    },
    reset: (state) => {
      state.value = 0;
    }
  }
});
```

```
export const { increment, decrement, incrementByAmount, res
et } = counterSlice.actions;
export default counterSlice.reducer;

// Using in component
import { useSelector, useDispatch } from 'react-redux';
import { increment, decrement, reset } from './store/counte
rSlice';

function Counter() {
  const count = useSelector((state) => state.counter.valu
e);
  const dispatch = useDispatch();

  return (
    <div>
      <p>Count: {count}</p>
      <button onClick={() => dispatch(increment())}>+</butt
on>
      <button onClick={() => dispatch(decrement())}>-</butt
on>
      <button onClick={() => dispatch(reset())}>Reset</butt
on>
    </div>
  );
}
```

Async Actions with createAsyncThunk

```
// store/userSlice.js
import { createSlice, createAsyncThunk } from '@reduxjs/too
lkit';

// Async thunk
export const fetchUser = createAsyncThunk(
  'user/fetchUser',
  async (userId) => {
```

```
    const response = await fetch(`/api/users/${userId}`);
    const data = await response.json();
    return data;
  }
);

const userSlice = createSlice({
  name: 'user',
  initialState: {
    data: null,
    loading: false,
    error: null
  },
  reducers: {
    clearUser: (state) => {
      state.data = null;
    }
  },
  extraReducers: (builder) => {
    builder
      .addCase(fetchUser.pending, (state) => {
        state.loading = true;
        state.error = null;
      })
      .addCase(fetchUser.fulfilled, (state, action) => {
        state.loading = false;
        state.data = action.payload;
      })
      .addCase(fetchUser.rejected, (state, action) => {
        state.loading = false;
        state.error = action.error.message;
      });
  }
});

export const { clearUser } = userSlice.actions;
export default userSlice.reducer;
```

```
// Using in component
function UserProfile({ userId }) {
  const dispatch = useDispatch();
  const { data, loading, error } = useSelector((state) => s
tate.user);

  useEffect(() => {
    dispatch(fetchUser(userId));
  }, [userId, dispatch]);

  if (loading) return <div>Loading...</div>;
  if (error) return <div>Error: {error}</div>;
  if (!data) return null;

  return (
    <div>
      <h1>{data.name}</h1>
      <p>{data.email}</p>
    </div>
  );
}
```

12. Zustand

Basic Store

```
// store/useStore.js
import { create } from 'zustand';

export const useStore = create((set) => ({
  count: 0,
  increment: () => set((state) => ({ count: state.count + 1
})),
  decrement: () => set((state) => ({ count: state.count - 1
})),
  reset: () => set({ count: 0 })
}));
```

```
// Using in component
import { useStore } from './store/useStore';

function Counter() {
  const count = useStore((state) => state.count);
  const increment = useStore((state) => state.increment);
  const decrement = useStore((state) => state.decrement);

  return (
    <div>
      <p>Count: {count}</p>
      <button onClick={increment}>+</button>
      <button onClick={decrement}>-</button>
    </div>
  );
}
```

Advanced Zustand Store

```
// store/useCartStore.js
import { create } from 'zustand';
import { persist } from 'zustand/middleware';

export const useCartStore = create(
  persist(
    (set, get) => ({
      items: [],

      addItem: (product) => set((state) => {
        const existingItem = state.items.find(item => item.
id === product.id);

        if (existingItem) {
          return {
            items: state.items.map(item =>
              item.id === product.id
                ? { ...item, quantity: item.quantity + 1 }
                : item
```

```
            )
          };
        }

        return {
          items: [...state.items, { ...product, quantity: 1
}]
        };
      }),

      removeItem: (productId) => set((state) => ({
        items: state.items.filter(item => item.id !== produ
ctId)
      })),

      updateQuantity: (productId, quantity) => set((state)
=> ({
        items: state.items.map(item =>
          item.id === productId ? { ...item, quantity } : i
tem
        )
      })),

      clearCart: () => set({ items: [] }),

      getTotalPrice: () => {
        const state = get();
        return state.items.reduce(
          (total, item) => total + (item.price * item.quant
ity),
          0
        );
      },

      getTotalItems: () => {
        const state = get();
        return state.items.reduce((total, item) => total +
item.quantity, 0);
```

```
      }
    }),
    {
      name: 'shopping-cart' // localStorage key
    }
  )
);

// Using in component
function ShoppingCart() {
  const items = useCartStore((state) => state.items);
  const removeItem = useCartStore((state) => state.removeIt
em);
  const getTotalPrice = useCartStore((state) => state.getTo
talPrice);

  return (
    <div>
      <h2>Shopping Cart</h2>
      {items.map(item => (
        <div key={item.id}>
          <span>{item.name} x {item.quantity}</span>
          <button onClick={() => removeItem(item.id)}>Remov
e</button>
        </div>
      ))}
      <p>Total: ${getTotalPrice()}</p>
    </div>
  );
}
```

13. React Query / TanStack Query

Setup

```
// main.jsx
import { QueryClient, QueryClientProvider } from '@tanstac
k/react-query';
```

```
import { ReactQueryDevtools } from '@tanstack/react-query-devtools';

const queryClient = new QueryClient({
  defaultOptions: {
    queries: {
      staleTime: 5 * 60 * 1000, // 5 minutes
      cacheTime: 10 * 60 * 1000 // 10 minutes
    }
  }
});

function App() {
  return (
    <QueryClientProvider client={queryClient}>
      <YourApp />
      <ReactQueryDevtools initialIsOpen={false} />
    </QueryClientProvider>
  );
}
```

Basic Query

```
import { useQuery } from '@tanstack/react-query';

function UserProfile({ userId }) {
  const {
    data,
    isLoading,
    isError,
    error,
    refetch
  } = useQuery({
    queryKey: ['user', userId],
    queryFn: async () => {
      const response = await fetch(`/api/users/${userId}`);
      if (!response.ok) throw new Error('Network response was not ok');
```

```
      return response.json();
    }
  });

  if (isLoading) return <div>Loading...</div>;
  if (isError) return <div>Error: {error.message}</div>;

  return (
    <div>
      <h1>{data.name}</h1>
      <p>{data.email}</p>
      <button onClick={refetch}>Refresh</button>
    </div>
  );
}
```

Mutations

```
import { useMutation, useQueryClient } from '@tanstack/react-query';

function CreatePost() {
  const queryClient = useQueryClient();

  const mutation = useMutation({
    mutationFn: async (newPost) => {
      const response = await fetch('/api/posts', {
        method: 'POST',
        headers: { 'Content-Type': 'application/json' },
        body: JSON.stringify(newPost)
      });
      return response.json();
    },
    onSuccess: () => {
      // Invalidate and refetch
      queryClient.invalidateQueries({ queryKey: ['posts'] });
    }
```

```
  });

  const handleSubmit = (e) => {
    e.preventDefault();
    mutation.mutate({
      title: 'New Post',
      content: 'Post content'
    });
  };

  return (
    <form onSubmit={handleSubmit}>
      <button type="submit" disabled={mutation.isLoading}>
        {mutation.isLoading ? 'Creating...' : 'Create Pos
t'}
      </button>
      {mutation.isError && <div>Error: {mutation.error.mess
age}</div>}
      {mutation.isSuccess && <div>Post created!</div>}
    </form>
  );
}
```

Pagination

```
function PostsList() {
  const [page, setPage] = useState(1);

  const { data, isLoading, isError } = useQuery({
    queryKey: ['posts', page],
    queryFn: async () => {
      const response = await fetch(`/api/posts?page=${page}
&limit=10`);
      return response.json();
    },
    keepPreviousData: true
  });
```

```
  return (
    <div>
      {isLoading && <div>Loading...</div>}
      {isError && <div>Error loading posts</div>}

      {data && (
        <>
          <div>
            {data.posts.map(post => (
              <PostCard key={post.id} post={post} />
            ))}
          </div>

          <div>
            <button
              onClick={() => setPage(old => Math.max(old -
1, 1))}
              disabled={page === 1}
            >
              Previous
            </button>
            <span>Page {page}</span>
            <button
              onClick={() => setPage(old => old + 1)}
              disabled={!data.hasMore}
            >
              Next
            </button>
          </div>
        </>
      )}
    </div>
  );
}
```

Infinite Queries

```
import { useInfiniteQuery } from '@tanstack/react-query';

function InfinitePostsList() {
  const {
    data,
    fetchNextPage,
    hasNextPage,
    isFetchingNextPage
  } = useInfiniteQuery({
    queryKey: ['posts'],
    queryFn: async ({ pageParam = 1 }) => {
      const response = await fetch(`/api/posts?page=${pageP
aram}`);
      return response.json();
    },
    getNextPageParam: (lastPage, pages) => {
      return lastPage.hasMore ? pages.length + 1 : undefine
d;
    }
  });

  return (
    <div>
      {data?.pages.map((page, i) => (
        <div key={i}>
          {page.posts.map(post => (
            <PostCard key={post.id} post={post} />
          ))}
        </div>
      ))}

      <button
        onClick={() => fetchNextPage()}
        disabled={!hasNextPage || isFetchingNextPage}
      >
        {isFetchingNextPage
          ? 'Loading more...'
          : hasNextPage
```

```
            ? 'Load More'
            : 'No more posts'}
        </button>
      </div>
    );
}
```

Component Patterns

14. Higher-Order Components (HOC)

```
// withAuth HOC
function withAuth(Component) {
  return function AuthenticatedComponent(props) {
    const { user, loading } = useAuth();

    if (loading) return <div>Loading...</div>;
    if (!user) return <Navigate to="/login" />;

    return <Component {...props} user={user} />;
  };
}

// Usage
function Dashboard({ user }) {
  return <div>Welcome, {user.name}!</div>;
}

export default withAuth(Dashboard);

// withLoading HOC
function withLoading(Component) {
  return function LoadingComponent({ isLoading, ...props })
{
    if (isLoading) return <div>Loading...</div>;
    return <Component {...props} />;
```

```
  };
}

// Usage
const UserListWithLoading = withLoading(UserList);

function App() {
  const [isLoading, setIsLoading] = useState(true);
  const [users, setUsers] = useState([]);

  return <UserListWithLoading isLoading={isLoading} users=
{users} />;
}
```

15. Render Props Pattern

```
// Mouse tracker component
function MouseTracker({ render }) {
  const [position, setPosition] = useState({ x: 0, y: 0 });

  useEffect(() => {
    const handleMouseMove = (e) => {
      setPosition({ x: e.clientX, y: e.clientY });
    };

    window.addEventListener('mousemove', handleMouseMove);
    return () => window.removeEventListener('mousemove', ha
ndleMouseMove);
  }, []);

  return render(position);
}

// Usage
function App() {
  return (
    <MouseTracker
```

```
      render={({ x, y }) => (
        <div>
          Mouse position: {x}, {y}
        </div>
      )}
    />
  );
}

// Data fetcher with render prop
function DataFetcher({ url, render }) {
  const [data, setData] = useState(null);
  const [loading, setLoading] = useState(true);

  useEffect(() => {
    fetch(url)
      .then(res => res.json())
      .then(data => {
        setData(data);
        setLoading(false);
      });
  }, [url]);

  return render({ data, loading });
}

// Usage
<DataFetcher
  url="/api/users"
  render={({ data, loading }) => (
    loading ? <div>Loading...</div> : <UserList users={dat
a} />
  )}
/>
```

16. Compound Components Pattern

```jsx
// Tabs component
const TabsContext = createContext();

function Tabs({ children, defaultValue }) {
  const [activeTab, setActiveTab] = useState(defaultValue);

  return (
    <TabsContext.Provider value={{ activeTab, setActiveTab
}}>
      <div className="tabs">{children}</div>
    </TabsContext.Provider>
  );
}

function TabList({ children }) {
  return <div className="tab-list">{children}</div>;
}

function Tab({ value, children }) {
  const { activeTab, setActiveTab } = useContext(TabsContex
t);
  const isActive = activeTab === value;

  return (
    <button
      className={`tab ${isActive ? 'active' : ''}`}
      onClick={() => setActiveTab(value)}
    >
      {children}
    </button>
  );
}

function TabPanels({ children }) {
  return <div className="tab-panels">{children}</div>;
}

function TabPanel({ value, children }) {
```

```
  const { activeTab } = useContext(TabsContext);

  if (activeTab !== value) return null;

  return <div className="tab-panel">{children}</div>;
}

// Compose the component
Tabs.List = TabList;
Tabs.Tab = Tab;
Tabs.Panels = TabPanels;
Tabs.Panel = TabPanel;

// Usage
function App() {
  return (
    <Tabs defaultValue="profile">
      <Tabs.List>
        <Tabs.Tab value="profile">Profile</Tabs.Tab>
        <Tabs.Tab value="settings">Settings</Tabs.Tab>
        <Tabs.Tab value="notifications">Notifications</Tab
s.Tab>
      </Tabs.List>

      <Tabs.Panels>
        <Tabs.Panel value="profile">
          <h2>Profile Content</h2>
        </Tabs.Panel>
        <Tabs.Panel value="settings">
          <h2>Settings Content</h2>
        </Tabs.Panel>
        <Tabs.Panel value="notifications">
          <h2>Notifications Content</h2>
        </Tabs.Panel>
      </Tabs.Panels>
    </Tabs>
  );
}
```

17. Controlled vs Uncontrolled Components

Controlled Components

```
// Controlled input - React controls the value
function ControlledForm() {
  const [name, setName] = useState('');
  const [email, setEmail] = useState('');

  const handleSubmit = (e) => {
    e.preventDefault();
    console.log({ name, email });
  };

  return (
    <form onSubmit={handleSubmit}>
      <input
        value={name}
        onChange={(e) => setName(e.target.value)}
        placeholder="Name"
      />
      <input
        value={email}
        onChange={(e) => setEmail(e.target.value)}
        placeholder="Email"
      />
      <button type="submit">Submit</button>
    </form>
  );
}
```

Uncontrolled Components

```
// Uncontrolled input - DOM controls the value
function UncontrolledForm() {
  const nameRef = useRef(null);
  const emailRef = useRef(null);
```

```
  const handleSubmit = (e) => {
    e.preventDefault();
    console.log({
      name: nameRef.current.value,
      email: emailRef.current.value
    });
  };

  return (
    <form onSubmit={handleSubmit}>
      <input ref={nameRef} placeholder="Name" />
      <input ref={emailRef} placeholder="Email" />
      <button type="submit">Submit</button>
    </form>
  );
}
```

React Router

18. Basic Routing

```
// App.jsx
import { BrowserRouter, Routes, Route, Link, Navigate } fro
m 'react-router-dom';

function App() {
  return (
    <BrowserRouter>
      <nav>
        <Link to="/">Home</Link>
        <Link to="/about">About</Link>
        <Link to="/users">Users</Link>
      </nav>

      <Routes>
        <Route path="/" element={<Home />} />
```

```
        <Route path="/about" element={<About />} />
        <Route path="/users" element={<Users />} />
        <Route path="/users/:id" element={<UserDetail />} /
>
        <Route path="*" element={<NotFound />} />
      </Routes>
    </BrowserRouter>
  );
}

function Home() {
  return <h1>Home Page</h1>;
}

function About() {
  return <h1>About Page</h1>;
}

function Users() {
  return (
    <div>
      <h1>Users</h1>
      <Link to="/users/1">User 1</Link>
      <Link to="/users/2">User 2</Link>
    </div>
  );
}

function NotFound() {
  return <h1>404 - Page Not Found</h1>;
}
```

19. Dynamic Routes & URL Parameters

```
import { useParams, useNavigate, useSearchParams } from 're
act-router-dom';
```

```jsx
// Dynamic route parameter
function UserDetail() {
  const { id } = useParams();
  const [user, setUser] = useState(null);

  useEffect(() => {
    fetch(`/api/users/${id}`)
      .then(res => res.json())
      .then(data => setUser(data));
  }, [id]);

  if (!user) return <div>Loading...</div>;

  return (
    <div>
      <h1>{user.name}</h1>
      <p>{user.email}</p>
    </div>
  );
}

// Programmatic navigation
function LoginForm() {
  const navigate = useNavigate();

  const handleLogin = async (credentials) => {
    await login(credentials);
    navigate('/dashboard');
    // Or with state: navigate('/dashboard', { state: { fro
m: 'login' } });
  };

  return <form onSubmit={handleLogin}>...</form>;
}

// Query parameters
function ProductsPage() {
  const [searchParams, setSearchParams] = useSearchParams
```

```
();
  const category = searchParams.get('category');
  const sort = searchParams.get('sort');

  const updateCategory = (newCategory) => {
    setSearchParams({ category: newCategory, sort });
  };

  return (
    <div>
      <p>Category: {category}</p>
      <p>Sort: {sort}</p>
      <button onClick={() => updateCategory('electronics')}
>
        Electronics
      </button>
    </div>
  );
}
```

20. Protected Routes

```
import { Navigate, Outlet } from 'react-router-dom';

// Protected route wrapper
function ProtectedRoute({ children }) {
  const { user, loading } = useAuth();

  if (loading) return <div>Loading...</div>;

  if (!user) {
    return <Navigate to="/login" replace />;
  }

  return children ? children : <Outlet />;
}
```

```jsx
// Usage
function App() {
  return (
    <Routes>
      <Route path="/login" element={<Login />} />

      {/* Protected routes */}
      <Route element={<ProtectedRoute />}>
        <Route path="/dashboard" element={<Dashboard />} />
        <Route path="/profile" element={<Profile />} />
        <Route path="/settings" element={<Settings />} />
      </Route>
    </Routes>
  );
}

// Role-based protection
function ProtectedRoute({ children, allowedRoles }) {
  const { user } = useAuth();

  if (!user) return <Navigate to="/login" />;

  if (allowedRoles && !allowedRoles.includes(user.role)) {
    return <Navigate to="/unauthorized" />;
  }

  return children;
}

// Usage
<Route
  path="/admin"
  element={
    <ProtectedRoute allowedRoles={['admin']}>
      <AdminPanel />
    </ProtectedRoute>
  }
/>
```

21. Nested Routes & Layouts

```
// Layout component
function Layout() {
  return (
    <div>
      <header>
        <nav>
          <Link to="/">Home</Link>
          <Link to="/dashboard">Dashboard</Link>
        </nav>
      </header>

      <main>
        <Outlet /> {/* Child routes render here */}
      </main>

      <footer>© 2024 My App</footer>
    </div>
  );
}

// Nested routes
function App() {
  return (
    <Routes>
      <Route path="/" element={<Layout />}>
        <Route index element={<Home />} />
        <Route path="about" element={<About />} />

        <Route path="dashboard" element={<DashboardLayout />
>}>
          <Route index element={<DashboardHome />} />
          <Route path="analytics" element={<Analytics />} />
>
          <Route path="reports" element={<Reports />} />
        </Route>
      </Route>
```

```
    </Routes>
  );
}
```

Next.js Rendering Strategies

22. Server-Side Rendering (SSR)

```
// pages/user/[id].js (Pages Router)
export async function getServerSideProps(context) {
  const { id } = context.params;

  // Fetch data on every request
  const res = await fetch(`https://api.example.com/users/
${id}`);
  const user = await res.json();

  return {
    props: {
      user
    }
  };
}

export default function UserPage({ user }) {
  return (
    <div>
      <h1>{user.name}</h1>
      <p>{user.email}</p>
    </div>
  );
}

// With authentication
export async function getServerSideProps(context) {
  const { req } = context;
```

```
  const token = req.cookies.token;

  if (!token) {
    return {
      redirect: {
        destination: '/login',
        permanent: false
      }
    };
  }

  const user = await fetchUser(token);

  return {
    props: { user }
  };
}
```

23. Static Site Generation (SSG)

```
// pages/blog/[slug].js
export async function getStaticPaths() {
  // Get list of all possible paths
  const posts = await fetch('https://api.example.com/post
s').then(r => r.json());

  const paths = posts.map(post => ({
    params: { slug: post.slug }
  }));

  return {
    paths,
    fallback: false // or 'blocking' or true
  };
}

export async function getStaticProps({ params }) {
```

```
  const { slug } = params;

  // Fetch data at build time
  const post = await fetch(`https://api.example.com/posts/
${slug}`)
    .then(r => r.json());

  return {
    props: {
      post
    },
    revalidate: 60 // ISR: Regenerate page every 60 seconds
  };
}

export default function BlogPost({ post }) {
  return (
    <article>
      <h1>{post.title}</h1>
      <div dangerouslySetInnerHTML={{ __html: post.content
}} />
    </article>
  );
}
```

24. Incremental Static Regeneration (ISR)

```
// pages/products/[id].js
export async function getStaticProps({ params }) {
  const product = await fetch(`https://api.example.com/prod
ucts/${params.id}`)
    .then(r => r.json());

  return {
    props: {
      product
    },
```

```
    revalidate: 10 // Revalidate every 10 seconds
  };
}

export async function getStaticPaths() {
  // Only pre-render most popular products
  const popularProducts = await fetch('https://api.example.
com/products/popular')
    .then(r => r.json());

  const paths = popularProducts.map(product => ({
    params: { id: product.id.toString() }
  }));

  return {
    paths,
    fallback: 'blocking' // Generate other pages on-demand
  };
}

export default function ProductPage({ product }) {
  return (
    <div>
      <h1>{product.name}</h1>
      <p>${product.price}</p>
    </div>
  );
}
```

25. Client-Side Rendering (CSR)

```
// pages/dashboard.js
import { useEffect, useState } from 'react';

export default function Dashboard() {
  const [data, setData] = useState(null);
  const [loading, setLoading] = useState(true);
```

```
  useEffect(() => {
    // Fetch data on client side
    fetch('/api/dashboard')
      .then(res => res.json())
      .then(data => {
        setData(data);
        setLoading(false);
      });
  }, []);

  if (loading) return <div>Loading...</div>;

  return (
    <div>
      <h1>Dashboard</h1>
      <p>Data: {JSON.stringify(data)}</p>
    </div>
  );
}

// Or with SWR
import useSWR from 'swr';

const fetcher = (url) => fetch(url).then(r => r.json());

export default function Dashboard() {
  const { data, error, isLoading } = useSWR('/api/dashboar
d', fetcher);

  if (isLoading) return <div>Loading...</div>;
  if (error) return <div>Error loading data</div>;

  return <div>Dashboard: {data.title}</div>;
}
```

Next.js App Router (v13+)

26. File-Based Routing

```
app/
├── page.js                    // Route: /
├── about/
│   └── page.js                // Route: /about
├── blog/
│   ├── page.js                // Route: /blog
│   └── [slug]/
│       └── page.js            // Route: /blog/[slug]
├── dashboard/
│   ├── layout.js              // Layout for dashboard
│   ├── page.js                // Route: /dashboard
│   ├── analytics/
│   │   └── page.js            // Route: /dashboard/analytic
s
│   └── settings/
│       └── page.js            // Route: /dashboard/settings
└── api/
    └── users/
        └── route.js           // API Route: /api/users
```

27. Server & Client Components

```
// app/page.js - Server Component (default)
async function getData() {
  const res = await fetch('https://api.example.com/data', {
    cache: 'no-store' // Disable caching for dynamic data
  });
  return res.json();
}

export default async function Page() {
  const data = await getData();

  return (
    <div>
```

```
      <h1>Server Component</h1>
      <p>{data.message}</p>
    </div>
  );
}

// app/components/Counter.js - Client Component
'use client';

import { useState } from 'react';

export default function Counter() {
  const [count, setCount] = useState(0);

  return (
    <div>
      <p>Count: {count}</p>
      <button onClick={() => setCount(count + 1)}>Increment
</button>
    </div>
  );
}

// Mixing Server and Client Components
// app/page.js
import Counter from './components/Counter';

async function getData() {
  const res = await fetch('https://api.example.com/data');
  return res.json();
}

export default async function Page() {
  const data = await getData(); // Server-side data fetchin
g

  return (
    <div>
```

```
      <h1>{data.title}</h1>
      <Counter /> {/* Client component */}
    </div>
  );
}
```

28. Layouts & Templates

```
// app/layout.js - Root Layout
export const metadata = {
  title: 'My App',
  description: 'App description'
};

export default function RootLayout({ children }) {
  return (
    <html lang="en">
      <body>
        <header>
          <nav>Navigation</nav>
        </header>
        <main>{children}</main>
        <footer>Footer</footer>
      </body>
    </html>
  );
}

// app/dashboard/layout.js - Nested Layout
export default function DashboardLayout({ children }) {
  return (
    <div className="dashboard-layout">
      <aside>
        <nav>
          <a href="/dashboard">Home</a>
          <a href="/dashboard/analytics">Analytics</a>
          <a href="/dashboard/settings">Settings</a>
```

```
        </nav>
      </aside>
      <div className="content">
        {children}
      </div>
    </div>
  );
}

// app/dashboard/template.js - Template (remounts on naviga
tion)
'use client';

export default function DashboardTemplate({ children }) {
  useEffect(() => {
    console.log('Template mounted');
  }, []);

  return <div className="template">{children}</div>;
}
```

29. Loading & Error Handling

```
// app/dashboard/loading.js
export default function Loading() {
  return (
    <div className="loading-spinner">
      <p>Loading dashboard...</p>
    </div>
  );
}

// app/dashboard/error.js
'use client';

export default function Error({ error, reset }) {
  return (
```

```
    <div>
      <h2>Something went wrong!</h2>
      <p>{error.message}</p>
      <button onClick={() => reset()}>Try again</button>
    </div>
  );
}

// app/dashboard/not-found.js
export default function NotFound() {
  return (
    <div>
      <h2>Page Not Found</h2>
      <p>Could not find the requested resource</p>
    </div>
  );
}

// Trigger not-found manually
import { notFound } from 'next/navigation';

export default async function Page({ params }) {
  const user = await getUser(params.id);

  if (!user) {
    notFound();
  }

  return <div>{user.name}</div>;
}
```

30. Dynamic Routes

```
// app/blog/[slug]/page.js
export default async function BlogPost({ params }) {
  const { slug } = params;
  const post = await getPost(slug);
```

```
  return (
    <article>
      <h1>{post.title}</h1>
      <div>{post.content}</div>
    </article>
  );
}

// Generate static params
export async function generateStaticParams() {
  const posts = await getPosts();

  return posts.map((post) => ({
    slug: post.slug
  }));
}

// Catch-all routes: app/docs/[...slug]/page.js
export default function Docs({ params }) {
  const { slug } = params; // slug is an array: ['getting-s
tarted', 'installation']

  return <div>Docs: {slug.join('/')}</div>;
}

// Optional catch-all: app/docs/[[...slug]]/page.js
// Matches /docs, /docs/a, /docs/a/b, etc.
```

31. Route Handlers (API Routes)

```
// app/api/users/route.js
import { NextResponse } from 'next/server';

// GET /api/users
export async function GET(request) {
  const { searchParams } = new URL(request.url);
```

```
  const page = searchParams.get('page') || 1;

  const users = await fetchUsers(page);

  return NextResponse.json({ users });
}

// POST /api/users
export async function POST(request) {
  const body = await request.json();
  const { name, email } = body;

  const newUser = await createUser({ name, email });

  return NextResponse.json({ user: newUser }, { status: 201
});
}

// Dynamic route: app/api/users/[id]/route.js
export async function GET(request, { params }) {
  const { id } = params;
  const user = await fetchUser(id);

  if (!user) {
    return NextResponse.json({ error: 'User not found' }, {
status: 404 });
  }

  return NextResponse.json({ user });
}

export async function PUT(request, { params }) {
  const { id } = params;
  const body = await request.json();

  const updatedUser = await updateUser(id, body);

  return NextResponse.json({ user: updatedUser });
```

```
}

export async function DELETE(request, { params }) {
  const { id } = params;
  await deleteUser(id);

  return NextResponse.json({ message: 'User deleted' });
}
```

Next.js Features

32. Image Optimization

```
import Image from 'next/image';

// Basic usage
export default function Profile() {
  return (
    <Image
      src="/profile.jpg"
      alt="Profile picture"
      width={500}
      height={500}
    />
  );
}

// External images
export default function Avatar() {
  return (
    <Image
      src="https://example.com/avatar.jpg"
      alt="Avatar"
      width={100}
      height={100}
    />
```

```
  );
}

// Configure domains in next.config.js
module.exports = {
  images: {
    domains: ['example.com', 'cdn.example.com']
  }
};

// Responsive images
<Image
  src="/hero.jpg"
  alt="Hero"
  fill
  style={{ objectFit: 'cover' }}
  priority // Load immediately
/>

// With placeholder
<Image
  src="/photo.jpg"
  alt="Photo"
  width={800}
  height={600}
  placeholder="blur"
  blurDataURL="..."
/>
```

33. Font Optimization

```
// app/layout.js
import { Inter, Roboto_Mono } from 'next/font/google';

const inter = Inter({
  subsets: ['latin'],
  display: 'swap'
```

```
});

const robotoMono = Roboto_Mono({
  subsets: ['latin'],
  display: 'swap',
  variable: '--font-roboto-mono'
});

export default function RootLayout({ children }) {
  return (
    <html lang="en" className={`${inter.className} ${roboto
Mono.variable}`}>
      <body>{children}</body>
    </html>
  );
}

// Custom/local fonts
import localFont from 'next/font/local';

const myFont = localFont({
  src: './my-font.woff2',
  display: 'swap'
});

export default function Layout({ children }) {
  return <div className={myFont.className}>{children}</div
>;
}
```

34. Metadata API (SEO)

```
// Static metadata
// app/page.js
export const metadata = {
  title: 'Home Page',
  description: 'Welcome to my app',
```

```
  keywords: ['nextjs', 'react', 'web development'],
  openGraph: {
    title: 'Home Page',
    description: 'Welcome to my app',
    images: ['/og-image.jpg']
  },
  twitter: {
    card: 'summary_large_image',
    title: 'Home Page',
    description: 'Welcome to my app',
    images: ['/twitter-image.jpg']
  }
};

export default function Page() {
  return <h1>Home</h1>;
}

// Dynamic metadata
// app/blog/[slug]/page.js
export async function generateMetadata({ params }) {
  const post = await getPost(params.slug);

  return {
    title: post.title,
    description: post.excerpt,
    openGraph: {
      title: post.title,
      description: post.excerpt,
      images: [post.coverImage]
    }
  };
}

export default function BlogPost({ params }) {
  // ...
}
```

35. Middleware

```
// middleware.js (in root directory)
import { NextResponse } from 'next/server';

export function middleware(request) {
  const token = request.cookies.get('token');
  const { pathname } = request.nextUrl;

  // Protect dashboard routes
  if (pathname.startsWith('/dashboard')) {
    if (!token) {
      return NextResponse.redirect(new URL('/login', reques
t.url));
    }
  }

  // Add custom header
  const response = NextResponse.next();
  response.headers.set('X-Custom-Header', 'value');

  return response;
}

export const config = {
  matcher: ['/dashboard/:path*', '/admin/:path*']
};

// Redirect based on locale
export function middleware(request) {
  const locale = request.cookies.get('locale') || 'en';
  const { pathname } = request.nextUrl;

  if (!pathname.startsWith(`/${locale}`)) {
    return NextResponse.redirect(
      new URL(`/${locale}${pathname}`, request.url)
    );
```

```
  }
}
```

36. Environment Variables

```
# .env.local
NEXT_PUBLIC_API_URL=https://api.example.com
DATABASE_URL=postgresql://user:pass@localhost:5432/db
SECRET_KEY=your-secret-key
```

```
// Usage in components (only NEXT_PUBLIC_* available in cli
ent)
export default function Component() {
  const apiUrl = process.env.NEXT_PUBLIC_API_URL;

  return <div>API: {apiUrl}</div>;
}

// Usage in API routes or server components (all variables
available)
export async function GET() {
  const dbUrl = process.env.DATABASE_URL;
  // Use database URL
}

// next.config.js
module.exports = {
  env: {
    CUSTOM_KEY: 'my-value'
  }
};
```

Data Fetching

37. Fetch API with Next.js

```
// Static data fetching
async function getData() {
  const res = await fetch('https://api.example.com/data', {
    cache: 'force-cache' // Default: SSG behavior
  });

  if (!res.ok) throw new Error('Failed to fetch');

  return res.json();
}

export default async function Page() {
  const data = await getData();
  return <div>{data.title}</div>;
}

// Dynamic data fetching
async function getData() {
  const res = await fetch('https://api.example.com/data', {
    cache: 'no-store' // SSR behavior
  });
  return res.json();
}

// Revalidate (ISR behavior)
async function getData() {
  const res = await fetch('https://api.example.com/data', {
    next: { revalidate: 60 } // Revalidate every 60 seconds
  });
  return res.json();
}
```

38. SWR (Stale-While-Revalidate)

```
import useSWR from 'swr';

const fetcher = (url) => fetch(url).then(r => r.json());
```

```
function Profile() {
  const { data, error, isLoading } = useSWR('/api/user', fe
tcher);

  if (error) return <div>Failed to load</div>;
  if (isLoading) return <div>Loading...</div>;

  return <div>Hello {data.name}!</div>;
}

// With options
function Profile() {
  const { data, error, mutate } = useSWR('/api/user', fetch
er, {
    refreshInterval: 3000, // Poll every 3 seconds
    revalidateOnFocus: true,
    revalidateOnReconnect: true
  });

  const updateUser = async () => {
    await fetch('/api/user', {
      method: 'PUT',
      body: JSON.stringify({ name: 'New Name' })
    });

    mutate(); // Revalidate data
  };

  return (
    <div>
      <p>{data.name}</p>
      <button onClick={updateUser}>Update</button>
    </div>
  );
}

// Pagination
```

```
function PostsList() {
  const [pageIndex, setPageIndex] = useState(0);
  const { data, error } = useSWR(`/api/posts?page=${pageIndex}`, fetcher);

  return (
    <div>
      {data && data.posts.map(post => (
        <PostCard key={post.id} post={post} />
      ))}

      <button onClick={() => setPageIndex(pageIndex - 1)}>Previous</button>
      <button onClick={() => setPageIndex(pageIndex + 1)}>Next</button>
    </div>
  );
}
```

39. Axios with React

```
import axios from 'axios';
import { useEffect, useState } from 'react';

// Create axios instance
const api = axios.create({
  baseURL: 'https://api.example.com',
  timeout: 5000,
  headers: {
    'Content-Type': 'application/json'
  }
});

// Add request interceptor
api.interceptors.request.use(
  (config) => {
    const token = localStorage.getItem('token');
```

```
    if (token) {
      config.headers.Authorization = `Bearer ${token}`;
    }
    return config;
  },
  (error) => Promise.reject(error)
);

// Add response interceptor
api.interceptors.response.use(
  (response) => response,
  (error) => {
    if (error.response?.status === 401) {
      // Handle unauthorized
      window.location.href = '/login';
    }
    return Promise.reject(error);
  }
);

// Usage in component
function UserProfile({ userId }) {
  const [user, setUser] = useState(null);
  const [loading, setLoading] = useState(true);
  const [error, setError] = useState(null);

  useEffect(() => {
    api.get(`/users/${userId}`)
      .then(response => {
        setUser(response.data);
        setLoading(false);
      })
      .catch(error => {
        setError(error.message);
        setLoading(false);
      });
  }, [userId]);
```

```
  if (loading) return <div>Loading...</div>;
  if (error) return <div>Error: {error}</div>;

  return <div>{user.name}</div>;
}

// POST request
async function createPost(data) {
  try {
    const response = await api.post('/posts', data);
    return response.data;
  } catch (error) {
    throw error;
  }
}

// Custom hook
function useApi(url) {
  const [data, setData] = useState(null);
  const [loading, setLoading] = useState(true);
  const [error, setError] = useState(null);

  useEffect(() => {
    let cancelled = false;

    api.get(url)
      .then(response => {
        if (!cancelled) {
          setData(response.data);
          setLoading(false);
        }
      })
      .catch(error => {
        if (!cancelled) {
          setError(error);
          setLoading(false);
        }
      });
```

```
    return () => {
      cancelled = true;
    };
  }, [url]);

  return { data, loading, error };
}
```

CSS & Styling

40. Tailwind CSS

```
// Installation
// npm install -D tailwindcss postcss autoprefixer
// npx tailwindcss init -p

// tailwind.config.js
module.exports = {
  content: [
    './pages/**/*.{js,jsx}',
    './components/**/*.{js,jsx}',
    './app/**/*.{js,jsx}'
  ],
  theme: {
    extend: {
      colors: {
        primary: '#3B82F6',
        secondary: '#10B981'
      },
      fontFamily: {
        sans: ['Inter', 'sans-serif']
      }
    }
  },
  plugins: []
```

```
};

// globals.css
@tailwind base;
@tailwind components;
@tailwind utilities;

// Component with Tailwind
export default function Button({ children, variant = 'prima
ry' }) {
  const baseClasses = 'px-4 py-2 rounded-lg font-medium tra
nsition-colors';
  const variantClasses = {
    primary: 'bg-blue-500 text-white hover:bg-blue-600',
    secondary: 'bg-gray-200 text-gray-800 hover:bg-gray-30
0',
    danger: 'bg-red-500 text-white hover:bg-red-600'
  };

  return (
    <button className={`${baseClasses} ${variantClasses[var
iant]}`}>
      {children}
    </button>
  );
}

// Responsive design
<div className="
  w-full
  sm:w-1/2
  md:w-1/3
  lg:w-1/4
  xl:w-1/5
">
  Responsive width
</div>
```

```
// Dark mode
<div className="bg-white dark:bg-gray-800 text-black dark:t
ext-white">
  Dark mode support
</div>

// Custom utilities
@layer utilities {
  .text-shadow {
    text-shadow: 2px 2px 4px rgba(0, 0, 0, 0.1);
  }
}
```

41. CSS Modules

```
/* Button.module.css */
.button {
  padding: 0.5rem 1rem;
  border-radius: 0.25rem;
  font-weight: 500;
  transition: all 0.2s;
}

.primary {
  background-color: #3B82F6;
  color: white;
}

.primary:hover {
  background-color: #2563EB;
}

.secondary {
  background-color: #E5E7EB;
  color: #1F2937;
}
```

```jsx
// Button.jsx
import styles from './Button.module.css';

export default function Button({ children, variant = 'prima
ry' }) {
  return (
    <button className={`${styles.button} ${styles[variant]}
`}>
      {children}
    </button>
  );
}

// Composition
<button className={`${styles.button} ${styles.large} ${styl
es.primary}`}>
  Click me
</button>
```

42. Styled Components

```jsx
// Installation: npm install styled-components

import styled from 'styled-components';

// Basic styled component
const Button = styled.button`
  padding: 0.5rem 1rem;
  border-radius: 0.25rem;
  font-weight: 500;
  background-color: #3B82F6;
  color: white;
  border: none;
  cursor: pointer;
  transition: background-color 0.2s;

  &:hover {
```

```
    background-color: #2563EB;
  }

  &:disabled {
    opacity: 0.5;
    cursor: not-allowed;
  }
`;

// With props
const Button = styled.button`
  padding: 0.5rem 1rem;
  background-color: ${props => props.primary ? '#3B82F6' :
'#E5E7EB'};
  color: ${props => props.primary ? 'white' : '#1F2937'};
`;

// Usage
<Button primary>Primary</Button>
<Button>Secondary</Button>

// Extending styles
const PrimaryButton = styled(Button)`
  background-color: #3B82F6;
  color: white;
`;

// Theme provider
import { ThemeProvider } from 'styled-components';

const theme = {
  colors: {
    primary: '#3B82F6',
    secondary: '#10B981'
  },
  spacing: {
    small: '0.5rem',
    medium: '1rem',
```

```
    large: '2rem'
  }
};

function App() {
  return (
    <ThemeProvider theme={theme}>
      <YourApp />
    </ThemeProvider>
  );
}

// Using theme
const Button = styled.button`
  background-color: ${props => props.theme.colors.primary};
  padding: ${props => props.theme.spacing.medium};
`;
```

Performance Optimization

43. React.memo

```
// Without React.memo - re-renders on every parent render
function ExpensiveComponent({ data }) {
  console.log('ExpensiveComponent rendered');
  return <div>{data}</div>;
}

// With React.memo - only re-renders when props change
const ExpensiveComponent = React.memo(function ExpensiveCom
ponent({ data }) {
  console.log('ExpensiveComponent rendered');
  return <div>{data}</div>;
});

// Custom comparison function
```

```
const ExpensiveComponent = React.memo(
  function ExpensiveComponent({ user }) {
    return <div>{user.name}</div>;
  },
  (prevProps, nextProps) => {
    // Return true if props are equal (don't re-render)
    return prevProps.user.id === nextProps.user.id;
  }
);
```

44. Code Splitting & Lazy Loading

```
import { lazy, Suspense } from 'react';

// Lazy load component
const HeavyComponent = lazy(() => import('./HeavyComponen
t'));

function App() {
  return (
    <div>
      <Suspense fallback={<div>Loading...</div>}>
        <HeavyComponent />
      </Suspense>
    </div>
  );
}

// Route-based code splitting
const Home = lazy(() => import('./pages/Home'));
const About = lazy(() => import('./pages/About'));
const Dashboard = lazy(() => import('./pages/Dashboard'));

function App() {
  return (
    <Suspense fallback={<LoadingSpinner />}>
      <Routes>
```

```
        <Route path="/" element={<Home />} />
        <Route path="/about" element={<About />} />
        <Route path="/dashboard" element={<Dashboard />} />
      </Routes>
    </Suspense>
  );
}

// Next.js dynamic imports
import dynamic from 'next/dynamic';

const DynamicComponent = dynamic(() => import('./HeavyCompo
nent'), {
  loading: () => <div>Loading...</div>,
  ssr: false // Disable server-side rendering
});

export default function Page() {
  return <DynamicComponent />;
}
```

45. Virtualization

```
// npm install react-window

import { FixedSizeList } from 'react-window';

function VirtualizedList({ items }) {
  const Row = ({ index, style }) => (
    <div style={style}>
      {items[index].name}
    </div>
  );

  return (
    <FixedSizeList
      height={600}
```

```
      itemCount={items.length}
      itemSize={50}
      width="100%"
    >
      {Row}
    </FixedSizeList>
  );
}

// Variable size list
import { VariableSizeList } from 'react-window';

function VariableList({ items }) {
  const getItemSize = (index) => items[index].height;

  const Row = ({ index, style }) => (
    <div style={style}>
      {items[index].content}
    </div>
  );

  return (
    <VariableSizeList
      height={600}
      itemCount={items.length}
      itemSize={getItemSize}
      width="100%"
    >
      {Row}
    </VariableSizeList>
  );
}
```

46. Image Optimization

```
// Lazy load images
function LazyImage({ src, alt }) {
```

```
  const imgRef = useRef(null);
  const [isVisible, setIsVisible] = useState(false);

  useEffect(() => {
    const observer = new IntersectionObserver(
      ([entry]) => {
        if (entry.isIntersecting) {
          setIsVisible(true);
          observer.disconnect();
        }
      },
      { threshold: 0.1 }
    );

    if (imgRef.current) {
      observer.observe(imgRef.current);
    }

    return () => observer.disconnect();
  }, []);

  return (
    <img
      ref={imgRef}
      src={isVisible ? src : ''}
      alt={alt}
      loading="lazy"
    />
  );
}

// Next.js Image optimization (already covered earlier)
import Image from 'next/image';

<Image
  src="/photo.jpg"
  alt="Photo"
  width={800}
```

```
  height={600}
  placeholder="blur"
  priority={false}
/>
```

Form Handling

47. React Hook Form

```
// npm install react-hook-form

import { useForm } from 'react-hook-form';

function RegistrationForm() {
  const {
    register,
    handleSubmit,
    formState: { errors },
    watch
  } = useForm();

  const onSubmit = (data) => {
    console.log(data);
  };

  const password = watch('password');

  return (
    <form onSubmit={handleSubmit(onSubmit)}>
      <input
        {...register('name', {
          required: 'Name is required',
          minLength: {
            value: 2,
            message: 'Name must be at least 2 characters'
          }
```

```
        })}
        placeholder="Name"
      />
      {errors.name && <span>{errors.name.message}</span>}

      <input
        {...register('email', {
          required: 'Email is required',
          pattern: {
            value: /^[A-Z0-9._%+-]+@[A-Z0-9.-]+\.[A-Z]{2,}
$/i,
            message: 'Invalid email address'
          }
        })}
        placeholder="Email"
      />
      {errors.email && <span>{errors.email.message}</span>}

      <input
        type="password"
        {...register('password', {
          required: 'Password is required',
          minLength: {
            value: 8,
            message: 'Password must be at least 8 character
s'
          }
        })}
        placeholder="Password"
      />
      {errors.password && <span>{errors.password.message}</
span>}

      <input
        type="password"
        {...register('confirmPassword', {
          required: 'Please confirm password',
          validate: value =>
```

```
            value === password || 'Passwords do not match'
          })}
          placeholder="Confirm Password"
        />
        {errors.confirmPassword && <span>{errors.confirmPassw
ord.message}</span>}

        <button type="submit">Register</button>
      </form>
    );
}
```

48. Form Validation with Zod

```
// npm install zod @hookform/resolvers

import { useForm } from 'react-hook-form';
import { zodResolver } from '@hookform/resolvers/zod';
import { z } from 'zod';

// Define schema
const schema = z.object({
  name: z.string().min(2, 'Name must be at least 2 characte
rs'),
  email: z.string().email('Invalid email address'),
  password: z.string().min(8, 'Password must be at least 8
characters'),
  age: z.number().min(18, 'Must be at least 18 years old'),
  terms: z.boolean().refine(val => val === true, {
    message: 'You must accept the terms'
  })
});

function RegistrationForm() {
  const {
    register,
    handleSubmit,
```

```
    formState: { errors }
  } = useForm({
    resolver: zodResolver(schema)
  });

  const onSubmit = (data) => {
    console.log(data);
  };

  return (
    <form onSubmit={handleSubmit(onSubmit)}>
      <input {...register('name')} placeholder="Name" />
      {errors.name && <span>{errors.name.message}</span>}

      <input {...register('email')} placeholder="Email" />
      {errors.email && <span>{errors.email.message}</span>}

      <input
        type="password"
        {...register('password')}
        placeholder="Password"
      />
      {errors.password && <span>{errors.password.message}</
span>}

      <input
        type="number"
        {...register('age', { valueAsNumber: true })}
        placeholder="Age"
      />
      {errors.age && <span>{errors.age.message}</span>}

      <label>
        <input type="checkbox" {...register('terms')} />
        I accept the terms
      </label>
      {errors.terms && <span>{errors.terms.message}</span>}
```

```
      <button type="submit">Register</button>
    </form>
  );
}
```

49. Formik

```
// npm install formik

import { Formik, Form, Field, ErrorMessage } from 'formik';
import * as Yup from 'yup';

const validationSchema = Yup.object({
  name: Yup.string()
    .min(2, 'Name must be at least 2 characters')
    .required('Name is required'),
  email: Yup.string()
    .email('Invalid email address')
    .required('Email is required'),
  password: Yup.string()
    .min(8, 'Password must be at least 8 characters')
    .required('Password is required')
});

function RegistrationForm() {
  const initialValues = {
    name: '',
    email: '',
    password: ''
  };

  const handleSubmit = (values, { setSubmitting }) => {
    console.log(values);
    setSubmitting(false);
  };

  return (
```

```
    <Formik
      initialValues={initialValues}
      validationSchema={validationSchema}
      onSubmit={handleSubmit}
    >
      {({ isSubmitting }) => (
        <Form>
          <Field name="name" placeholder="Name" />
          <ErrorMessage name="name" component="div" />

          <Field name="email" type="email" placeholder="Ema
il" />
          <ErrorMessage name="email" component="div" />

          <Field name="password" type="password" placeholde
r="Password" />
          <ErrorMessage name="password" component="div" />

          <button type="submit" disabled={isSubmitting}>
            {isSubmitting ? 'Submitting...' : 'Submit'}
          </button>
        </Form>
      )}
    </Formik>
  );
}
```

TypeScript with React

50. Basic TypeScript Setup

```
// Component with props
interface ButtonProps {
  text: string;
  onClick: () => void;
  variant?: 'primary' | 'secondary';
```

```
  disabled?: boolean;
}

export function Button({ text, onClick, variant = 'primar
y', disabled = false }: ButtonProps) {
  return (
    <button
      onClick={onClick}
      disabled={disabled}
      className={`btn-${variant}`}
    >
      {text}
    </button>
  );
}

// Usage
<Button text="Click me" onClick={() => console.log('Clicke
d')} />
```

51. TypeScript with Hooks

```
import { useState, useEffect, useRef } from 'react';

// useState with type inference
function Counter() {
  const [count, setCount] = useState(0); // Type inferred a
s number
  const [name, setName] = useState(''); // Type inferred as
string

  return (
    <div>
      <p>{count}</p>
      <p>{name}</p>
    </div>
  );
```

```
}

// useState with explicit type
interface User {
  id: number;
  name: string;
  email: string;
}

function UserProfile() {
  const [user, setUser] = useState<User | null>(null);

  useEffect(() => {
    fetchUser().then(data => setUser(data));
  }, []);

  return <div>{user?.name}</div>;
}

// useRef with types
function TextInput() {
  const inputRef = useRef<HTMLInputElement>(null);

  const focusInput = () => {
    inputRef.current?.focus();
  };

  return (
    <>
      <input ref={inputRef} />
      <button onClick={focusInput}>Focus</button>
    </>
  );
}

// Custom hook with TypeScript
function useFetch<T>(url: string) {
  const [data, setData] = useState<T | null>(null);
```

```
  const [loading, setLoading] = useState(true);
  const [error, setError] = useState<string | null>(null);

  useEffect(() => {
    fetch(url)
      .then(res => res.json())
      .then((data: T) => {
        setData(data);
        setLoading(false);
      })
      .catch(err => {
        setError(err.message);
        setLoading(false);
      });
  }, [url]);

  return { data, loading, error };
}

// Usage
interface Post {
  id: number;
  title: string;
  content: string;
}

function PostsList() {
  const { data, loading, error } = useFetch<Post[]>('/api/p
osts');

  if (loading) return <div>Loading...</div>;
  if (error) return <div>Error: {error}</div>;

  return (
    <div>
      {data?.map(post => (
        <div key={post.id}>{post.title}</div>
      ))}
```

```
    </div>
  );
}
```

52. Event Types

```
function Form() {
  const handleInputChange = (e: React.ChangeEvent<HTMLInput
Element>) => {
    console.log(e.target.value);
  };

  const handleSelectChange = (e: React.ChangeEvent<HTMLSele
ctElement>) => {
    console.log(e.target.value);
  };

  const handleSubmit = (e: React.FormEvent<HTMLFormElement
>) => {
    e.preventDefault();
    console.log('Form submitted');
  };

  const handleClick = (e: React.MouseEvent<HTMLButtonElemen
t>) => {
    console.log('Button clicked');
  };

  const handleKeyPress = (e: React.KeyboardEvent<HTMLInputE
lement>) => {
    if (e.key === 'Enter') {
      console.log('Enter pressed');
    }
  };

  return (
    <form onSubmit={handleSubmit}>
```

```
      <input onChange={handleInputChange} onKeyPress={handl
eKeyPress} />
      <select onChange={handleSelectChange}>
        <option>Option 1</option>
      </select>
      <button onClick={handleClick}>Submit</button>
    </form>
  );
}
```

53. Generic Components

```
interface ListProps<T> {
  items: T[];
  renderItem: (item: T) => React.ReactNode;
}

function List<T>({ items, renderItem }: ListProps<T>) {
  return (
    <ul>
      {items.map((item, index) => (
        <li key={index}>{renderItem(item)}</li>
      ))}
    </ul>
  );
}

// Usage
interface User {
  id: number;
  name: string;
}

function App() {
  const users: User[] = [
    { id: 1, name: 'John' },
    { id: 2, name: 'Jane' }
```

```
  ];

  return (
    <List
      items={users}
      renderItem={(user) => <span>{user.name}</span>}
    />
  );
}
```

Testing

54. Jest & React Testing Library

```
// npm install --save-dev jest @testing-library/react @test
ing-library/jest-dom

// Button.jsx
export function Button({ onClick, children }) {
  return <button onClick={onClick}>{children}</button>;
}

// Button.test.jsx
import { render, screen, fireEvent } from '@testing-librar
y/react';
import { Button } from './Button';

describe('Button', () => {
  test('renders button with text', () => {
    render(<Button>Click me</Button>);
    expect(screen.getByText('Click me')).toBeInTheDocument
();
  });

  test('calls onClick when clicked', () => {
    const handleClick = jest.fn();
```

```
    render(<Button onClick={handleClick}>Click me</Button
>);

    fireEvent.click(screen.getByText('Click me'));
    expect(handleClick).toHaveBeenCalledTimes(1);
  });
});

// Testing async operations
test('loads and displays user', async () => {
  render(<UserProfile userId={1} />);

  expect(screen.getByText('Loading...')).toBeInTheDocument
();

  const userName = await screen.findByText('John Doe');
  expect(userName).toBeInTheDocument();
});

// Testing with user interactions
import userEvent from '@testing-library/user-event';

test('submits form', async () => {
  const user = userEvent.setup();
  const handleSubmit = jest.fn();

  render(<LoginForm onSubmit={handleSubmit} />);

  await user.type(screen.getByLabelText('Email'), 'test@exa
mple.com');
  await user.type(screen.getByLabelText('Password'), 'passw
ord123');
  await user.click(screen.getByRole('button', { name: 'Logi
n' }));

  expect(handleSubmit).toHaveBeenCalledWith({
    email: 'test@example.com',
    password: 'password123'
```

```
  });
});
```

55. Mocking API Calls

```
// Using jest.mock
jest.mock('./api', () => ({
  fetchUser: jest.fn()
}));

import { fetchUser } from './api';

test('fetches and displays user', async () => {
  fetchUser.mockResolvedValue({
    id: 1,
    name: 'John Doe',
    email: 'john@example.com'
  });

  render(<UserProfile userId={1} />);

  const userName = await screen.findByText('John Doe');
  expect(userName).toBeInTheDocument();
  expect(fetchUser).toHaveBeenCalledWith(1);
});

// Mock fetch globally
global.fetch = jest.fn(() =>
  Promise.resolve({
    json: () => Promise.resolve({ name: 'John' })
  })
);

test('fetches data', async () => {
  render(<Component />);

  await screen.findByText('John');
```

```
  expect(global.fetch).toHaveBeenCalledWith('/api/user');
});
```

Build Tools & Deployment

56. Vite

```
# Create Vite project
npm create vite@latest my-app -- --template react
cd my-app
npm install
npm run dev
```

```
// vite.config.js
import { defineConfig } from 'vite';
import react from '@vitejs/plugin-react';

export default defineConfig({
  plugins: [react()],
  server: {
    port: 3000,
    proxy: {
      '/api': {
        target: 'http://localhost:8000',
        changeOrigin: true
      }
    }
  },
  build: {
    outDir: 'dist',
    sourcemap: true
  }
});
```

57. ESLint & Prettier

```
// .eslintrc.json
{
  "extends": [
    "eslint:recommended",
    "plugin:react/recommended",
    "plugin:react-hooks/recommended"
  ],
  "rules": {
    "react/prop-types": "off",
    "no-unused-vars": "warn"
  }
}

// .prettierrc
{
  "semi": true,
  "singleQuote": true,
  "tabWidth": 2,
  "trailingComma": "es5"
}
```

58. Deployment

```
# Build for production
npm run build

# Vercel deployment
npm install -g vercel
vercel

# Netlify deployment
npm install -g netlify-cli
netlify deploy --prod

# Docker
# Dockerfile
FROM node:18-alpine
```

```
WORKDIR /app
COPY package*.json ./
RUN npm install
COPY . .
RUN npm run build
EXPOSE 3000
CMD ["npm", "start"]
```

Interview Preparation Topics

Common Interview Questions

1. What is Virtual DOM?

The Virtual DOM is a lightweight copy of the actual DOM. React uses it to optimize updates by:

1. Creating a virtual representation of the UI
2. Comparing it with the previous version (diffing)
3. Updating only the changed parts in the real DOM (reconciliation)

2. Explain React Lifecycle

Class components have lifecycle methods like componentDidMount, componentDidUpdate, componentWillUnmount. In functional components, these are handled by useEffect.

3. What is the difference between state and props?

- Props: Read-only data passed from parent to child
- State: Mutable data managed within the component

4. What is prop drilling and how to avoid it?

Prop drilling is passing props through multiple levels of components. Solutions:

- Context API
- State management libraries (Redux, Zustand)
- Component composition

5. Controlled vs Uncontrolled components?

- Controlled: Form data handled by React state
- Uncontrolled: Form data handled by the DOM

6. What is useEffect cleanup function?

The cleanup function runs before the component unmounts or before the effect runs again, used to clean up subscriptions, timers, etc.

7. Explain useMemo and useCallback

- useMemo: Memoizes a computed value
- useCallback: Memoizes a function reference

8. What is Redux and when to use it?

Redux is a state management library for predictable state containers. Use it when:

- Multiple components need the same state
- State updates are complex
- You need middleware for side effects

9. Server-side Rendering (SSR) vs Client-side Rendering (CSR)?

- SSR: HTML rendered on server, better SEO, faster initial load
- CSR: HTML rendered in browser, more interactive, slower initial load

10. What is Next.js and its advantages?

Next.js is a React framework providing:

- SSR and SSG out of the box
- File-based routing
- API routes
- Image optimization
- Built-in CSS support

Additional Resources

Recommended Learning Path

1. **Week 1-2: React Fundamentals**
    - Components, Props, State
    - Hooks (useState, useEffect)
    - Event handling
2. **Week 3-4: Advanced React**
    - useContext, useReducer
    - Custom hooks
    - Performance optimization
3. **Week 5-6: State Management**
    - Context API deep dive
    - Redux Toolkit basics
    - React Query
4. **Week 7-8: Next.js**
    - Pages vs App Router
    - Data fetching strategies
    - API routes
5. **Week 9-10: Real Projects**
    - E-commerce site
    - Dashboard application
    - Blog with CMS

Practice Projects

1. **Todo App** - State management, CRUD operations
2. **Weather App** - API integration, async operations
3. **E-commerce** - Cart, checkout, state management

4. **Dashboard** - Charts, data visualization, authentication
5. **Blog** - Next.js SSG, MDX, CMS integration

Best Practices

- Keep components small and focused
- Use TypeScript for type safety
- Write tests for critical functionality
- Optimize performance with memoization
- Follow consistent naming conventions
- Use ESLint and Prettier
- Document complex logic
- Handle errors gracefully
- Implement proper loading states
- Make components accessible (a11y)

Good luck with your interview preparation! 🚀